MicroAlpha Engine

A C++/Python Research Engine for Short-Horizon Limit Order Book Prediction

"An approximate answer to the right problem is worth a good deal more than an exact answer to an approximate problem."
- John Tukey

by

Gautier Petit

github.com/gautierpetit
linkedin.com/in/gautierpetitch

May, 2026

Copyright (c) 2026 Gautier Petit
For research/education only. Not investment advice.
This document is licensed under the Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License (CC BY-NC-ND 4.0).

Table of Contents

I. Executive Summary: 5
II. Motivation and Scope: 6
Research focus 6
Methodological discipline 6
Engineering design 6
Empirical questions 6
Scope boundaries 7
III. System Architecture and Research Pipeline 8
Architecture overview 8
Compiled feature layer 8
Experiment orchestration 8
Chronological splitting and pooling 9
Artifacts, modules, and reproducibility 9
IV. Data 11
Sample coverage 11
Sample composition 11
Tie prevalence 11
Data limitations 11
Data availability 12
V. Feature Engineering 13
Feature families 13
Best-level order flow and liquidity imbalance 13
Depth and price-pressure features 14
Temporal aggregation features 15
Implementation 15
VI. Labeling Design and Task Formulation 16
Forward midprice change 16
Directional formulations 16
Movement formulation 17

Implementation and interpretation 17
VII. Modeling and Experimental Design 18
Model scope 18
Reference models 18
Parameterization 18
Chronological splitting 18
Multi-ticker pooling 19
Evaluation framework 19
Artifacts and reproducibility 19
VIII. Main Results 20
Reference pooled result 20
Cross-sectional heterogeneity 20
Label dependence and diagnostic overview 21
Horizon sensitivity 22
Movement prediction 22
Unconditional direction robustness 22
Summary 23
IX. Model Interpretation 24
Interpretation framework 24
Shared core signal 24
Logistic regression 24
HistGradientBoosting 25
Weak features 26
Interpretation summary 26
X. Limitations 27
Data access and sample coverage 27
Prediction rather than execution 27
Cross-sectional interpretation 27
Model scope 28
Scope of conclusions 28

XI. Extensions and Future Work 29
Broader data coverage 29
Cross-sectional heterogeneity 29
Task and horizon analysis 29
Modeling extensions 29
Future research path 30
XII. Conclusion 31
XIII. Appendix 32
A. Reference Configuration 32
B. Feature Definitions 33
C. Artifact Structure 34
D. Additional Diagnostics 36

I. Executive Summary:

MicroAlpha Engine is a research project on short-horizon limit order book prediction built around an event-driven, reproducible experimental pipeline. It examines whether near-term midprice direction can be inferred from contemporaneous order book state and recent event flow, using a compact set of microstructure features computed in C++ and orchestrated in Python. The resulting framework combines market microstructure analysis, modular pipeline design, and reproducible experiment management.

The empirical setting uses LOBSTER sample data for five NASDAQ names, AAPL, AMZN, GOOG, INTC, and MSFT, all drawn from the same trading day and represented at 10 depth levels. The reference experiment uses a 500-event forecast horizon with the direction task under the binary_drop_ties labeling mode. In this formulation, zero forward midprice changes are removed, so the task corresponds to direction prediction conditional on a future price move.

Two models are used throughout the main analysis: logistic regression as a linear benchmark and HistGradientBoostingClassifier as a modest nonlinear tabular model. In the pooled five-ticker reference run, the models achieve test ROC AUC values of 0.679 and 0.686 respectively. These aggregate results mask substantial heterogeneity across names, which becomes clear in per-ticker evaluation. In the same pooled-trained setting, AAPL, AMZN, and GOOG remain close to the low-0.5 AUC range, while INTC and MSFT are materially stronger, with AUCs around 0.81 and 0.73.

A central finding is that task definition materially shapes the interpretation of predictability. Under the main direction setup, tie rates differ sharply across names: they are very low for AAPL, AMZN, and GOOG, but much higher for INTC and MSFT. This makes the filtered direction task meaningfully easier for some names than for others. Additional diagnostic experiments, including movement prediction and keeping ties as class 0, indicate that tie filtering amplifies the effect but does not fully account for it. The resulting picture is one of persistent cross-sectional heterogeneity rather than a uniform pooling effect.

Interpretation results are stable across models. The dominant features are best-level queue imbalance, microprice deviation, and short-horizon accumulated OFI, while the nonlinear model draws additional information from depth imbalance, short-term volatility, and selected normalized OFI windows. These results are consistent with standard microstructure intuition and anchor the empirical findings in interpretable book dynamics.

From a systems perspective, the work is organized as a modular research engine with typed configuration, C++ feature computation through pybind11, time-respecting ticker-level splits, pooled evaluation, automated artifact generation, and a preserved reference run. It provides a structured framework for analyzing short-horizon order book predictability under explicit modeling and data constraints.

II. Motivation and Scope:

Research focus

MicroAlpha Engine was developed as a focused study in short-horizon market microstructure prediction and research-system design. It centers on a simple question: how much directional information is contained in event-level limit order book state and recent order flow, and how can that question be studied within a clean, reproducible framework. The emphasis is therefore placed equally on empirical analysis and implementation discipline. The project adopts a compact and interpretable setup built around event-driven labeling, a restrained feature set grounded in standard microstructure concepts, two well-understood model classes, and a modular experiment pipeline.

Methodological discipline

A second motivation was methodological. Short-horizon order book prediction is highly sensitive to implementation details that are easy to understate in less structured workflows. Event-time versus clock-time framing, feature-label alignment, tie handling, and train-test separation all affect the effective learning problem. For that reason, the project was designed as a pipeline: each ticker is loaded independently, features are computed deterministically, labels are created from forward midprice changes, time splits are applied per ticker, and pooled datasets are formed only after ticker-level splitting. This keeps the mechanics of the workflow explicit and makes the resulting empirical findings easier to interpret.

Engineering design

The project also has a clear engineering motivation. Event-level market data creates a natural setting for combining lower-level numerical computation with a higher-level research workflow. In MicroAlpha Engine, all production features are computed in C++, exposed to Python through pybind11, and then integrated into a Python pipeline responsible for configuration, training, evaluation, diagnostics, and artifact generation. This division of labor supports both clarity and reproducibility: performance-sensitive feature computation is isolated in a compiled component, while the broader experiment logic remains readable and easy to inspect.

Empirical questions

At the empirical level, the project addresses four main questions. First, it studies whether short-horizon midprice direction is predictable from a compact set of standard microstructure features derived from the top ten levels of the order book. Second, it examines whether pooling data across multiple tickers improves model performance relative to single-name training. Third, it investigates how strongly predictability differs across names within the same sample. Fourth, it evaluates how the formulation of the prediction task, especially the treatment of zero forward returns, changes the

effective difficulty and interpretation of the problem. These questions emerged directly from the research process and define the analytical scope of the report.

Scope boundaries

The project also has clear non-goals. It does not attempt to model execution, queue position, latency, costs, routing, or inventory dynamics, since those issues would require a substantially broader framework and would change the nature of the study. It does not present a live trading framework or a production deployment architecture. It also does not attempt to exhaust the model space, to optimize aggressively over hyperparameters, or to position itself as a deep learning benchmark. Given the scale of the available data, broad model expansion would add complexity more quickly than it would add reliable inference. These boundaries keep the project aligned with its central purpose: the study of event-driven order book prediction under explicit data, modeling, and implementation constraints.

III. System Architecture and Research Pipeline

Architecture overview

MicroAlpha Engine is organized as a modular research pipeline with a clear separation between feature computation, experiment orchestration, model training, evaluation, and artifact generation. The project is built around a two-layer architecture: a compiled C++ feature engine for event-level numerical computation, and a Python pipeline for configuration, data loading, labeling, training, diagnostics, and reproducibility. Feature computation over high-frequency order book arrays is handled in a low-level component, while the broader experimental workflow remains in a higher-level research environment.

Compiled feature layer

The C++ layer computes the full production feature matrix from aligned arrays of bid prices, ask prices, bid sizes, ask sizes, midprice, and event timestamps. The implementation returns fourteen event-level features, including best-level OFI, normalized OFI, queue imbalance, depth imbalance at several levels, spread, microprice deviation, rolling OFI aggregates, rolling midprice volatility, and event intensity. The Python feature module acts as a thin interface over this compiled component, passing configuration-defined window parameters and exposing a stable feature contract to the rest of the pipeline.

The compiled component is exposed to Python through pybind11. The binding layer validates array dimensionality and shape consistency before dispatching computation to the underlying feature engine. The build process is managed through CMake, with the extension compiled as a C++20 module and copied into the Python package directory after build. This keeps the research workflow simple at runtime while preserving an explicit native build path.

Experiment orchestration

The Python layer is responsible for experiment orchestration. Configuration is loaded from YAML into typed dataclasses defining dataset paths, task settings, label mode, feature windows, split parameters, and model hyperparameters. In the reference experiment, the configuration specifies five tickers, ten order book levels, a price scale of 10,000, a 500-event horizon, the `direction` task under `binary_drop_ties`, a 70/30 train-test split, and fixed parameters for both logistic regression and HistGradientBoosting. The configuration is also serialized into the run artifact folder so that each experiment preserves an exact parameter snapshot.

The execution path begins with LOBSTER data loading. For each ticker, the loader reads the message file and the corresponding order book file, verifies that row counts match, checks that the number of order book columns is consistent with the configured depth, reconstructs bid and ask

price and size arrays, and computes the best-level midprice series. Validation also includes checks for non-finite values and crossed best quotes. The pipeline then computes the event-level feature matrix in C++ and constructs labels from the forward midprice change over the chosen event horizon. The label system supports both `direction` and `movement` tasks. For the `direction` task, the code supports two binary label modes: `binary_drop_ties`, which removes zero forward returns, and `binary_keep_ties_as_zero`, which retains them as class 0. For the `movement` task, the binary label is defined as whether the forward midprice change is nonzero. Features and labels are then aligned explicitly by trimming the final H observations and applying any task-specific validity mask.

Chronological splitting and pooling

A central design choice in the pipeline is that time splitting is performed independently for each ticker before any pooling occurs. Each ticker dataset is first built in isolation and then split into train and test segments using a strict time-ordered split with no shuffling. If pooled training is requested, the per-ticker train segments are concatenated together, and the same is done for the per-ticker test segments. This preserves temporal structure within each name and avoids cross-sectional contamination between train and test. It also makes the single-ticker case a direct special case of the same pipeline.

This split-first, pool-second structure also supports per-ticker evaluation within pooled experiments. The pipeline keeps track of the boundaries of each ticker's test segment inside the pooled test array, which makes it possible to evaluate pooled-trained models both at the aggregate level and at the individual ticker level. That capability became important in the later empirical analysis, since the pooled headline result concealed substantial heterogeneity across names. Model training is performed in the Python layer using a standardized logistic regression pipeline and a HistGradientBoostingClassifier. These are trained on the pooled or single-ticker training set returned by the pipeline, and evaluation is then conducted on the corresponding test set, with outputs including accuracy, ROC AUC, confusion matrices, ROC curves, score distributions, logistic coefficients, and permutation feature importance. Per-ticker pooled metrics are also generated when multiple tickers are used.

Artifacts, modules, and reproducibility

A further part of the architecture is the artifact and reproducibility layer. Each run is assigned a timestamped identifier and writes its outputs into a dedicated artifact directory. The saved outputs include the configuration snapshot, overall metrics, split summary, ticker summaries, feature diagnostics, pooled per-ticker metrics, JSON and CSV feature-importance tables, figures, trained models, and a run log. This provides a stable record of each experiment and makes completed runs inspectable without re-execution. In the saved reference run, the log records dataset construction

by ticker, tie rates, pooled split sizes, model training, evaluation metrics, and artifact generation in sequence.

The codebase is correspondingly divided into focused modules. Configuration is defined in `config.py`, data loading in `io.py`, feature interfacing in `features.py`, label construction in `labels.py`, dataset build and split logic in `pipeline.py`, model training and persistence in `models.py`, evaluation and plotting in `evaluation.py`, and flattened summaries in `diagnostics.py`. The experiment runner is `scripts/run_experiment.py`, which acts as a thin orchestration layer calling these components in sequence. The implementation is also supported by a targeted test suite covering configuration loading, LOBSTER loading, label alignment, pooled split consistency, feature-matrix and feature-name consistency, and smoke import of the compiled extension. The project includes Docker support as well, with code and configuration copied into the image and raw data plus artifact directories mounted from the host system. Together, these components provide a stable execution environment for the reference pipeline.

Overall, the system architecture is designed to make the empirical workflow explicit, inspectable, and reproducible. The later conclusions on pooling, tie prevalence, task definition, and cross-sectional heterogeneity depend on that structure. In that sense, the architecture is not separate from the research results, but part of the framework that makes their interpretation possible.

IV. Data

Sample coverage

The empirical study in MicroAlpha Engine uses LOBSTER sample limit order book data for five NASDAQ names: AAPL, AMZN, GOOG, INTC, and MSFT. In the reference configuration, all five tickers are drawn from June 21, 2012 and represented at 10 levels of depth. Each ticker is loaded from a pair of files consisting of an event message stream and the corresponding order book states. Prices are scaled by a factor of 10,000, consistent with the raw input representation used in the pipeline configuration.

Sample composition

The main reference run shows substantial variation in event counts across names. On 2012-06-21, AAPL contains 400,391 events, AMZN 269,748, GOOG 147,916, INTC 624,040, and MSFT 668,765. After label construction and feature-label alignment, the final dataset sizes also differ materially across tickers, ranging from 146,467 observations for GOOG to 538,985 for MSFT under the main `direction` and `binary_drop_ties` setup. These differences are relevant for interpretation because pooled training and pooled evaluation reflect both per-name predictability and sample composition.

Tie prevalence

A central empirical feature of the data is the uneven prevalence of zero forward midprice changes across names. Under the main `direction` task with `binary_drop_ties` at a 500-event horizon, tie rates are very low for AAPL, AMZN, and GOOG, but materially higher for INTC and MSFT. In the reference summaries, the tie rates are approximately 1.03% for AAPL, 1.52% for AMZN, 0.64% for GOOG, 27.12% for INTC, and 19.35% for MSFT.

At the pooled level, the reference experiment contains 1,260,547 training observations and 540,238 test observations after ticker-level construction and splitting, with similar class balances across both partitions. This gives the project enough scale to support a structured comparison of feature behavior, label design, pooled training, and per-ticker evaluation within the chosen sample. At the same time, the scope of inference remains limited by the coverage of the underlying data.

Data limitations

The project uses sample data from a single day and five tickers only. A broader dataset covering more names, more dates, and more market conditions would strengthen the empirical side of the study. This constraint affects the breadth of the conclusions that can be drawn, especially across regimes and over time, and it also supports the use of a restrained modeling framework rather than

broader model expansion. It does not, however, change the nature of the work carried out in the project. The pipeline, feature construction, labeling framework, evaluation logic, and interpretation remain valid as a research-engineering exercise under explicit data constraints.

Data availability

The repository also does not distribute the raw LOBSTER files, as code availability and market-data licensing are distinct issues. Instead, it distributes the code, configuration, diagnostics, and a preserved example run. The saved run artifacts allow the structure and outputs of the pipeline to be inspected directly, even though the raw proprietary inputs are not included.

For the purposes of this report, the data should therefore be understood in a precise way: it is sufficient to support a serious event-driven microstructure case study, but not broad enough to justify general claims beyond the sample examined here. The conclusions that follow are presented on that basis.

V. Feature Engineering

Feature families

The feature layer in MicroAlpha Engine consists of fourteen event-level variables derived from contemporaneous order book state and recent event flow. All features are computed in **C++** and exposed to Python through a single binding interface. The set is organized into two groups: contemporaneous order book state features and short-horizon temporal summaries.

The contemporaneous state features are:
`ofi_best`, `ofi_best_norm`, `queue_imbalance_best`, `depth_imbalance_3`, `depth_imbalance_5`, `depth_imbalance_10`, `spread`, and `microprice_deviation`.

The temporal features are:
`ofi_roll_sum_50`, `ofi_best_norm_roll_sum_10`, `ofi_best_norm_roll_sum_50`, `ofi_best_norm_roll_sum_100`, `midprice_vol_50`, and `event_intensity_1s`.

These variables are intended to represent a compact set of standard microstructure mechanisms: local liquidity imbalance, price pressure at the best quotes, depth asymmetry, recent signed order flow, short-horizon volatility, and event activity.

Best-level order flow and liquidity imbalance

At event t, let $b_t^{(k)}$ and $a_t^{(k)}$ denote the bid and ask prices at depth level k, and let $q_{b,t}^{(k)}$ and $q_{a,t}^{(k)}$ denote the corresponding displayed queue sizes. The best-level midprice is defined as

$$m_t = \frac{b_t^{(1)} + a_t^{(1)}}{2}.$$

Best-level order flow imbalance is constructed from changes in best bid and ask prices and queue sizes between consecutive events. Define the bid-side contribution:

$$\Delta B_t = \begin{cases} q_{b,t}^{(1)} & \text{if } b_t^{(1)} > b_{t-1}^{(1)} \\ q_{b,t}^{(1)} - q_{b,t-1}^{(1)} & \text{if } b_t^{(1)} = b_{t-1}^{(1)} \\ -q_{b,t-1}^{(1)} & \text{if } b_t^{(1)} < b_{t-1}^{(1)} \end{cases}$$

and the ask-side contribution:

$$\Delta A_t = \begin{cases} q_{a,t}^{(1)} & \text{if } a_t^{(1)} < a_{t-1}^{(1)} \\ q_{a,t}^{(1)} - q_{a,t-1}^{(1)} & \text{if } a_t^{(1)} = a_{t-1}^{(1)} \\ -q_{a,t-1}^{(1)} & \text{if } a_t^{(1)} > a_{t-1}^{(1)} \end{cases}$$

Then best-level OFI is:

$$\mathrm{OFI}_t = \Delta B_t - \Delta A_t$$

and therefore:

$$\mathrm{ofi_best}_t = \mathrm{OFI}_t$$

A normalized version is also included:

$$\mathrm{ofi_best_norm}_t = \frac{\mathrm{OFI}_t}{q_{b,t}^{(1)} + q_{a,t}^{(1)}}$$

Best-level queue imbalance is defined as:

$$\texttt{queue_imbalance_best}_t = \frac{q_{b,t}^{(1)}}{q_{b,t}^{(1)} + q_{a,t}^{(1)}}.$$

Together, these features summarize signed pressure and liquidity asymmetry at the top of the book.

Depth and price-pressure features

Depth imbalance extends the same idea beyond the best level. For a depth K, it is defined as:

$$\texttt{depth_imbalance_K}_t = \frac{\sum_{k=1}^{K} q_{b,t}^{(k)} - \sum_{k=1}^{K} q_{a,t}^{(k)}}{\sum_{k=1}^{K} q_{b,t}^{(k)} + \sum_{k=1}^{K} q_{a,t}^{(k)}}, \qquad K \in \{3,5,10\}.$$

The bid-ask spread is:

$$\texttt{spread}_t = a_t^{(1)} - b_t^{(1)}.$$

A second price-pressure feature is the deviation between the size-weighted best-quote price and the simple midprice. The size-weighted best-quote price, or microprice, is:

$$\mu_t = \frac{a_t^{(1)} q_{b,t}^{(1)} + b_t^{(1)} q_{a,t}^{(1)}}{q_{b,t}^{(1)} + q_{a,t}^{(1)}}.$$

The feature `microprice_deviation` is then:

$$\texttt{microprice_deviation}_t = \mu_t - m_t.$$

These features describe local price pressure and how displayed liquidity is distributed across the two sides of the book.

Temporal aggregation features

The temporal features extend the static state description by summarizing recent local history. The first is a rolling accumulation of best-level OFI over a 50-event window:

$$\texttt{ofi_roll_sum_50}_t = \sum_{j=t-49}^{t} \mathrm{OFI}_j.$$

The normalized OFI features apply the same rolling construction to the normalized best-level OFI over windows of 10, 50, and 100 events:

$$\texttt{ofi_best_norm_roll_sum_W}_t = \sum_{j=t-W+1}^{t} \texttt{ofi_best_norm}_j, \qquad W \in \{10,50,100\}.$$

Short-horizon volatility is measured from event-to-event midprice changes over a 50-event rolling window:

$$\texttt{midprice_vol_50}_t = \mathrm{sd}\,(m_j - m_{j-1})_{j=t-49}^{t}.$$

Event intensity is defined as the number of events arriving within the most recent one-second window:

$$\texttt{event_intensity_1s}_t = |\,\{j \le t : \tau t - \tau j \le 1s\}\,|$$

where τ_t denotes the event timestamp.

These temporal features allow the model to incorporate recent order flow, local state persistence, and event activity without moving to a sequential model architecture.

Implementation

All production features are computed in the compiled C++ layer. The Python feature module serves only as an interface: it passes validated arrays and configured window parameters to the extension, then returns the resulting matrix under a fixed feature contract. This contract is explicit in both code and tests. The feature-name generator in Python defines the expected column order, and the test suite verifies that the compiled output shape matches that specification.

At the implementation level, the C++ engine uses contiguous storage, explicit row-major indexing, and rolling accumulator structures for OFI sums and rolling volatility. Event intensity is computed from timestamps through a moving-window count. The binding layer validates dimensionality and shape compatibility across all inputs before dispatching computation. The resulting feature layer provides a consistent event-level representation of order book state and recent event flow for the downstream labeling, modeling, and evaluation steps.

VI. Labeling Design and Task Formulation

Forward midprice change

Label construction is a central part of the empirical design in MicroAlpha Engine. In short-horizon limit order book prediction, the target is not uniquely defined by the data alone. It depends on how future price change is measured, over what horizon it is evaluated, and how zero-return observations are treated. For that reason, the project treats labeling as part of the research question rather than as a fixed preprocessing step.

All labels in the project are derived from the forward midprice change over an event horizon H. Given a midprice series m_t, the pipeline computes the forward delta

$$\Delta m_t = m_{t+H} - m_t,$$

where H is measured in events, not in clock time. This choice is consistent with the event-driven structure of the project and ensures that each label is associated with a fixed amount of order book evolution rather than a variable amount of market activity.

The label system supports two task families: `direction` and `movement`. These tasks are built from the same forward delta series, but they encode different prediction questions.

Directional formulations

For the `direction` task, the project supports two binary label modes.

Under `binary_drop_ties`, observations with $\Delta m_t = 0$ are removed. The remaining labels are defined as:

$$y_t = \begin{cases} 1 & \text{if } \Delta m_t > 0 \\ 0 & \text{if } \Delta m_t < 0 \end{cases}$$

This formulation is simple, but its interpretation is important. Once zero-return observations are excluded, the model is no longer solving unconditional direction prediction. It is solving direction prediction conditional on a price move occurring over the horizon.

Under `binary_keep_ties_as_zero`, all observations are retained and labels are defined as:

$$y_t = \begin{cases} 1 & \text{if } \Delta m_t > 0 \\ 0 & \text{if } \Delta m_t \leq 0 \end{cases}$$

This provides a binary directional formulation that keeps zero-return events in the sample. It is not equivalent to the conditional setup above, and in practice it produces a different effective problem, especially for names with high tie prevalence. In the project, this formulation is used primarily as a robustness check on the interpretation of the main `binary_drop_ties` results.

Movement formulation

For the `movement` task, the label is defined as whether the forward midprice change is nonzero:

$$y_t = \begin{cases} 1 & \text{if } \Delta m_t \neq 0 \\ 0 & \text{if } \Delta m_t = 0 \end{cases}$$

This task isolates a different question from directional prediction: not where price moves, but whether price moves at all over the chosen horizon. The movement formulation is included to separate two effects that are otherwise entangled in the direction results: movement predictability itself and direction predictability conditional on movement.

The choice between these formulations has substantive consequences for interpretation. In particular, the apparent difficulty of the directional task depends not only on the feature set and model class, but also on how zero forward returns are treated in the underlying data.

Implementation and interpretation

From an implementation standpoint, the labeling system is compact and explicit. The project computes the forward delta once, derives task-specific labels from that common series, records summary diagnostics such as tie rate and final sample size, and then aligns the feature matrix to the valid label index set. This structure keeps the task definition transparent while allowing different label formulations to be compared without changing the rest of the pipeline.

The labeling design therefore serves two functions in the project. It defines the supervised learning target, and it provides one of the main analytical levers for interpreting the empirical behavior of the system.

VII. Modeling and Experimental Design

Model scope

The modeling framework in MicroAlpha Engine is built around a small number of controlled design choices. The experimental design remains deliberately narrow given the scale of the available data. Short-horizon limit order book prediction is a low signal-to-noise problem, and the present study is based on five tickers from a single trading day. Under these conditions, broader model exploration, aggressive hyperparameter search, or deep learning architectures would add complexity more quickly than they would add reliable inference. The modeling layer is therefore kept compact and aligned with the scope of the study. The objective is controlled comparison under explicit data constraints rather than frontier benchmarking across broader model classes.

Reference models

The project uses two classifiers throughout the main analysis: logistic regression and HistGradientBoostingClassifier. Logistic regression serves as the linear baseline. In the implementation, it is fitted within a pipeline that applies standardization before classification, reflecting the scale differences across the engineered features. HistGradientBoostingClassifier serves as a modest nonlinear tabular model able to capture interactions and threshold effects that a linear classifier cannot represent directly. The second model is included as a limited nonlinear comparison under the same experimental conditions.

Parameterization

Model parameters are fixed in configuration. Logistic regression uses standard L2 regularization with lbfgs optimization and an elevated iteration cap. HistGradientBoostingClassifier uses a learning rate of 0.05, 200 boosting iterations, 31 maximum leaf nodes, and 50 minimum samples per leaf. These settings provide a stable nonlinear benchmark without expanding the study into extensive hyperparameter search.

Chronological splitting

The experimental workflow is built around strict time-respecting splits. For each ticker, the feature matrix and label vector are split chronologically using a 70/30 train-test partition with no shuffling. This preserves the temporal ordering of the event stream and avoids contamination between training and evaluation periods. The split function records the split index, partition sizes, and positive-class rates for each run.

Multi-ticker pooling

When multiple tickers are included, each ticker is processed independently through loading, feature construction, labeling, and time-based splitting. Pooling occurs only after the per-ticker split has been completed. The pooled training set is formed by concatenating all ticker-level training segments, and the pooled test set is formed in the same way from the ticker-level test segments. This preserves chronological separation within each name and keeps the single-ticker case as a direct special case of the same pipeline.

This split-first, pool-second structure also supports per-ticker evaluation within pooled experiments. Because the pooled test set is assembled from ordered ticker-specific segments, the pipeline can reconstruct the location of each ticker inside the pooled test array and evaluate pooled-trained models both at the aggregate level and at the individual ticker level.

Evaluation framework

Evaluation is carried out using standard binary classification metrics. At the aggregate level, the project records accuracy and ROC AUC for each trained model. It also saves confusion matrices, ROC curves, and score-distribution plots. For pooled experiments, the same evaluation framework is applied at the per-ticker level inside the pooled test set. This provides a common basis for comparing the linear and nonlinear models, the pooled and single-ticker cases, and the alternative task formulations considered later in the report.

Artifacts and reproducibility

The modeling workflow is also integrated with the project's artifact and reproducibility layer. Each run preserves the experiment configuration, metrics, split summaries, per-ticker summaries, pooled per-ticker evaluation tables, plots, and logs inside a timestamped artifact folder. This keeps the experimental design inspectable both in code and in the saved outputs of the pipeline.

VIII. Main Results

Reference pooled result

The main empirical results are organized around the final pooled reference specification and a small set of diagnostic runs used to test the interpretation of that result. The reference run uses the direction task with binary_drop_ties at an event horizon of 500, trained on pooled ticker-level chronological splits across AAPL, AMZN, GOOG, INTC, and MSFT. Under this specification, the pooled dataset contains 1,260,547 training observations and 540,238 test observations, with stable positive-class rates across train and test at approximately 46.6%.

Under the reference specification, the pooled results show a clear predictive signal at the aggregate level. Logistic regression achieves a test accuracy of 0.626 and ROC AUC of 0.679, while HistGradientBoostingClassifier achieves accuracy of 0.623 and ROC AUC of 0.686. The nonlinear model therefore improves slightly on the linear baseline in ROC AUC, but the gap remains limited. This pattern suggests that the feature layer contains meaningful short-horizon information, and that some of that information is nonlinear, but the difference between the two model classes remains measured rather than large.

Cross-sectional heterogeneity

The pooled headline result, however, conceals substantial cross-sectional heterogeneity. When the pooled-trained models are evaluated separately on each ticker's test segment, AAPL remains weak, with AUC values of 0.517 for logistic regression and 0.535 for HistGradientBoosting. AMZN improves somewhat, reaching 0.544 and 0.559, and GOOG remains in a similar range at 0.533 and 0.559. By contrast, INTC and MSFT are materially stronger. INTC reaches 0.815 under logistic regression and 0.803 under HistGradientBoosting, while MSFT reaches 0.732 and 0.735. The pooled result is therefore not a uniform cross-asset uplift. It reflects a mixture of modest transfer across names and persistent differences in task difficulty across the sample.

This per-ticker decomposition changes the interpretation of the pooled result. Read in isolation, the aggregate AUC near 0.68 could suggest that pooling broadly resolves the weakness of single-name prediction. The ticker-level results show something more specific. The weaker names do not all become strong under pooling, and the stronger names remain strong even when evaluated separately inside the pooled experiment. The pooled headline improvement is therefore partly a transfer effect and partly a composition effect.

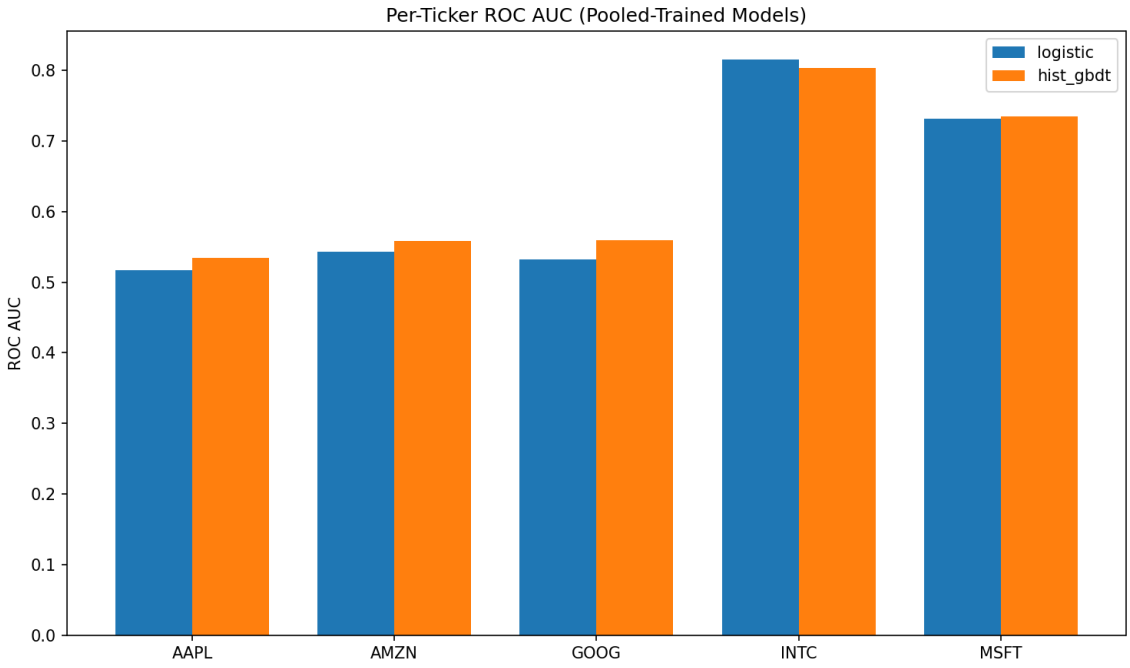

Figure 1. Per ticker ROC AUC Comparison

Label dependence and diagnostic overview

A central part of this interpretation concerns the treatment of zero forward returns. The reference specification uses `binary_drop_ties`, which removes observations with zero forward midprice change and therefore turns the task into direction prediction conditional on movement. In the reference summaries, tie rates differ sharply across names: approximately 1.03% for AAPL, 1.52% for AMZN, 0.64% for GOOG, 27.12% for INTC, and 19.35% for MSFT. This means that the formal label definition is shared across names, but the effective problem is not. For high-tie names, removing ties changes the sample materially and makes the conditional direction task more selective.

Table 1. Pooled performance summary across main and diagnostic specifications

Task	Label mode	Horizon	Logistic Accuracy	Logistic ROC AUC	HistGBDT Accuracy	HistGBDT ROC AUC
`direction`	`binary_drop_ties`	500	0.626	0.679	0.623	0.686
`direction`	`binary_drop_ties`	2000	0.566	0.568	0.546	0.526
`movement`	`binary`	500	0.829	0.746	0.824	0.744
`direction`	`binary_keep_ties_as_zero`	500	0.634	0.652	0.636	0.669

The repository includes one preserved reference run as a public example artifact, while the additional diagnostic runs discussed below are summarized in the report for robustness rather than bundled as separate example runs.

Horizon sensitivity

The first diagnostic check is the longer-horizon run at $H = 2000$, still under the `direction` task with `binary_drop_ties`. At this horizon, pooled performance drops sharply: logistic regression falls to 0.568 ROC AUC, and HistGradientBoosting falls to 0.526 ROC AUC. The stronger names also weaken materially. INTC falls from roughly 0.815 to 0.609 under logistic regression, and MSFT falls from 0.732 to 0.595. At the same time, tie rates decline for the high-tie names, with INTC falling from about 27.1% to 13.1% and MSFT from 19.3% to 12.0%. This result is consistent with the interpretation that part of the strong $H = 500$ result is linked to the conditional nature of the task under high tie prevalence rather than to a uniform gain from pooling alone.

Movement prediction

The second diagnostic check is the `movement` task at $H = 500$, which asks whether price moves at all rather than in which direction it moves. At the pooled level, this run produces high accuracy and relatively strong aggregate AUC, with logistic regression at 0.829 accuracy and 0.746 ROC AUC, and HistGradientBoosting at 0.824 accuracy and 0.744 ROC AUC. However, the per-ticker results show that accuracy is heavily influenced by class prevalence. For AAPL, AMZN, and GOOG, the test move rate is between roughly 98% and 99.6%, and reported accuracy is almost identical to that base rate. In those cases, a classifier can achieve very high accuracy simply by predicting movement almost all the time, which limits the usefulness of accuracy as a summary statistic. Their movement AUCs are only 0.560, 0.502, and 0.516 under logistic regression. Even for INTC and MSFT, movement AUC remains modest at 0.581 and 0.511 respectively. This clarifies an important point: the strongest results in the reference direction setup are not primarily driven by strong prediction of movement itself. They arise more from predicting direction on the subset of events where movement occurs.

Unconditional direction robustness

The third diagnostic check is the unconditional directional formulation `binary_keep_ties_as_zero` at $H = 500$. This provides the most direct robustness test against the main `binary_drop_ties` setup. At the pooled level, performance declines, but does not collapse. Logistic regression falls from 0.679 to 0.652 ROC AUC, and HistGradientBoosting falls from 0.686 to 0.669. The effect is more pronounced in the high-tie names. INTC declines from 0.815 / 0.803 under `binary_drop_ties` to 0.722 / 0.715 under `binary_keep_ties_as_zero`, while MSFT declines from 0.732 / 0.735 to 0.693 / 0.704. By contrast, AAPL, AMZN, and GOOG remain weak in both formulations, with only limited changes. This result gives a clear quantitative conclusion:

tie filtering amplifies the apparent predictability of the stronger names, but it does not fully create it. INTC and MSFT remain the strongest names even when ties are retained in the sample.

Summary

Taken together, the main results support four conclusions. First, the pooled reference specification contains a real short-horizon predictive signal, with aggregate ROC AUC close to 0.68 under both models. Second, that pooled result masks substantial heterogeneity across names. Third, the treatment of zero forward returns is not a secondary implementation detail, but part of the empirical problem definition. Fourth, the strongest cross-sectional effects cannot be reduced to pooling alone, movement prediction alone, or tie filtering alone.

A more complete explanation of the phenomenon would require broader date coverage, more systematic cross-sectional analysis, and explicit testing of microstructure characteristics such as spread regime, tick discreteness, queue structure, volatility, and event intensity across names. Within the present sample, the explanation can be narrowed substantially, but not fully resolved.

IX. Model Interpretation

Interpretation framework

Model interpretation in MicroAlpha Engine is based on two complementary outputs from the reference pooled run. For logistic regression, the project records fitted coefficients from the standardized model. For both logistic regression and HistGradientBoostingClassifier, it computes permutation feature importance on the held-out test set using ROC AUC as the scoring metric with 10 repeats. Taken together, these two views provide a consistent picture of which parts of the feature layer are driving the main results.

Shared core signal

The most important finding is that both models rely on a stable core set of microstructure features. Across coefficient magnitudes and permutation importance, three variables consistently dominate: `queue_imbalance_best`, `microprice_deviation`, and `ofi_roll_sum_50`. This concentration is useful for interpretation because it indicates that the predictive signal is not broadly distributed across many weak inputs. Instead, it is anchored in a small number of first-order quantities tied to local liquidity imbalance, size-weighted price pressure, and recent signed order flow.

Logistic regression

For logistic regression, the signal is especially concentrated. Permutation importance ranks `queue_imbalance_best` first by a wide margin, with a mean ROC AUC decrease of approximately 0.143 when permuted.

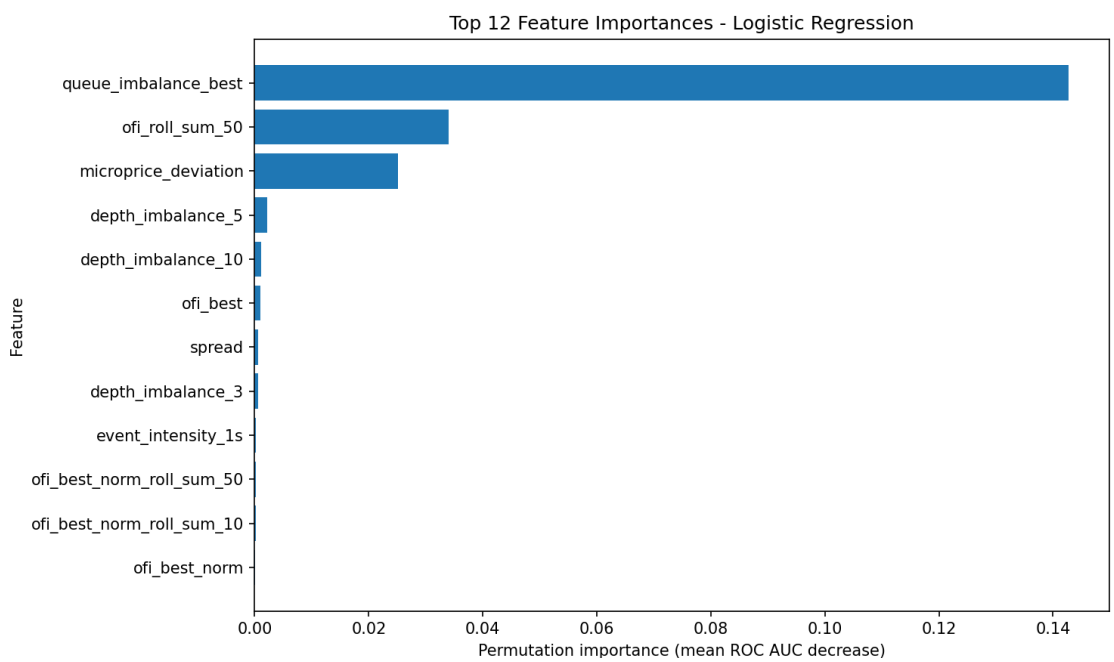

Figure 2. Feature importance of logistic regression model

The next two variables, `ofi_roll_sum_50` and `microprice_deviation`, are materially smaller at roughly 0.034 and 0.025 respectively. The fitted coefficients tell the same story. After standardization, `queue_imbalance_best` has the largest coefficient in absolute value at about 0.538, followed by `microprice_deviation` at about -0.341 and `ofi_roll_sum_50` at about 0.185. In other words, the linear model depends primarily on a narrow core signal, with best-level queue imbalance clearly dominant.

HistGradientBoosting

The HistGradientBoosting model uses the same core variables, but with a somewhat broader distribution of importance across the feature set. In the permutation-importance ranking, `microprice_deviation` is first at approximately 0.058, followed by `ofi_roll_sum_50` at 0.018 and `queue_imbalance_best` at 0.017. The next tier includes `midprice_vol_50`, `depth_imbalance_3`, `depth_imbalance_5`, and `spread`, all of which contribute more meaningfully than they do in the linear model. This suggests that the nonlinear model is extracting additional structure from the surrounding state variables while still relying on the same first-order microstructure core.

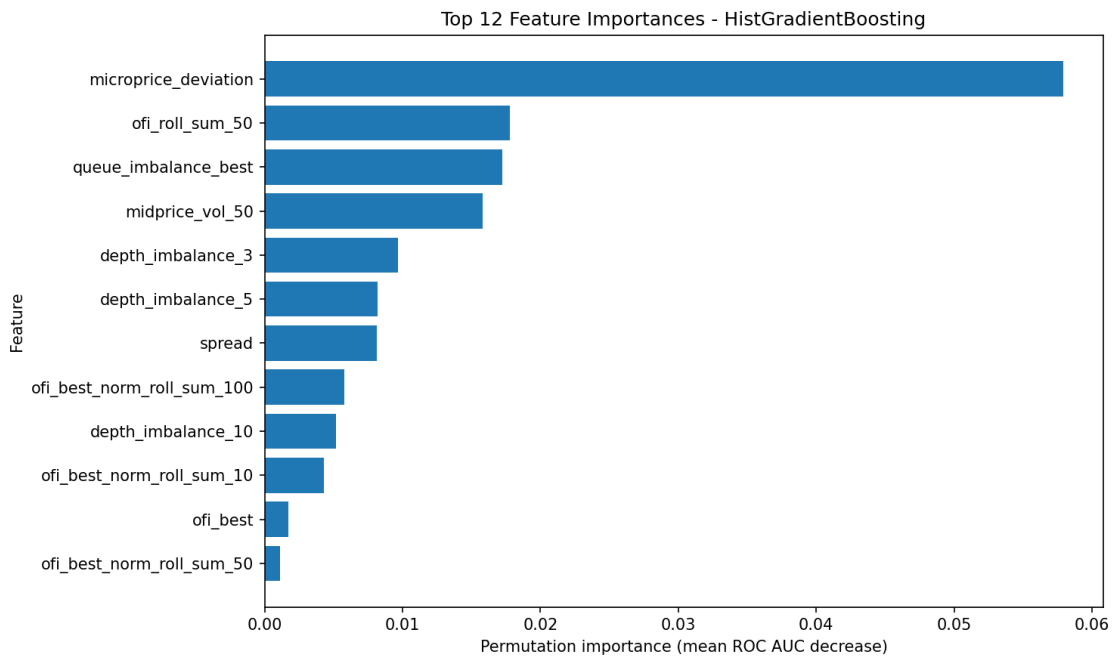

Figure 3. Feature Importance of HistGradientBoosting model

This contrast between models is informative. Logistic regression places most of its weight on a small number of highly local signals, especially best-level queue imbalance. HistGradientBoosting still depends on that same local information, but it appears better able to make use of interactions involving depth imbalance, short-term volatility, and selected normalized OFI windows. The difference is therefore not that the nonlinear model discovers an entirely different signal family. Rather, it extends the same base signal into a somewhat richer tabular structure.

Weak features

Several features contribute little in both models. In the logistic permutation-importance results, `ofi_best_norm`, `ofi_best_norm_roll_sum_100`, and `midprice_vol_50` are close to zero or slightly negative. In the HistGradientBoosting results, `ofi_best_norm`, `event_intensity_1s`, and `ofi_best_norm_roll_sum_50` are similarly weak relative to the leading variables. This does not imply that these features are useless in every setting, but within the reference specification they do not carry the same explanatory weight as the core liquidity and short-horizon order-flow variables.

Table 2. Top 5 features per model

Model	Feature	Importance mean	Importance std
HistGradientBoosting	`microprice_deviation`	0.05790	0.000345
	`ofi_roll_sum_50`	0.01780	0.000239
	`queue_imbalance_best`	0.01722	0.000302
	`midprice_vol_50`	0.01580	0.000369
	`depth_imbalance_3`	0.00965	0.000117
Logistic Regression	`queue_imbalance_best`	0.14268	0.000748
	`ofi_roll_sum_50`	0.03397	0.000311
	`microprice_deviation`	0.02509	0.000264
	`depth_imbalance_5`	0.00223	0.000119
	`depth_imbalance_10`	0.00116	0.000129

Interpretation summary

The interpretation that emerges is economically coherent. `queue_imbalance_best` measures the local balance of displayed liquidity at the best quotes. `microprice_deviation` captures whether size-weighted best-quote pressure pulls away from the simple midpoint. `ofi_roll_sum_50` summarizes recent signed pressure from order book updates. These are standard ways of representing short-horizon book pressure, and their dominance is consistent with the intended microstructure interpretation of the feature set.

The interpretation section does not prove a structural mechanism on its own, but it shows that the learned signal is concentrated in features consistent with standard microstructure intuition. Overall, the model-interpretation results support a restrained conclusion: the reference result is driven primarily by a stable core of best-level imbalance, local price pressure, and recent order-flow information, with the nonlinear model extracting some additional value from depth- and volatility-related structure around that core.

X. Limitations

Data access and sample coverage

The main limitation of MicroAlpha Engine is data coverage. The empirical study uses LOBSTER sample data for five NASDAQ tickers from a single trading day, all at ten levels of depth. This is sufficient to support event-driven feature construction, label analysis, pooled training, and cross-sectional comparison within the sample. It is not sufficient to support broad claims across time, regimes, or market conditions.

This limitation is partly practical. High-quality limit order book data is expensive, access-controlled, and subject to licensing restrictions. For an independent research project, broad historical coverage across many names and dates is not a neutral implementation choice, but a material data-access constraint. A larger dataset covering more dates, more names, and more market conditions would strengthen the empirical side of the study substantially.

The repository also does not distribute the raw LOBSTER files, as code availability and market-data licensing are distinct issues. It distributes the code, configuration, diagnostics, and a preserved reference run, but complete end-to-end rerunning of the pipeline requires the user to supply the same external market data inputs.

Prediction rather than execution

A second limitation is that the project studies prediction only, not execution. The pipeline models short-horizon directional and movement outcomes from order book state, but it does not simulate queue position, order placement, fill uncertainty, latency, transaction costs, routing, or inventory constraints. As a result, the reported metrics should be interpreted as classification results under explicit label definitions, not as evidence of deployable trading performance.

This boundary is deliberate. A proper execution-aware framework would require substantially more machinery than the current study: order placement logic, queue-state modeling, fill assumptions, latency modeling, transaction-cost assumptions, and realistic trading constraints. Adding a superficial execution layer would weaken rather than strengthen the project, because it would create the appearance of trading realism without the infrastructure required to support it. The present report therefore stops at predictive modeling and treats execution as a separate research problem.

Cross-sectional interpretation

A third limitation is that the explanation of the strongest cross-sectional effect remains incomplete. The project narrows the phenomenon substantially: the stronger performance of INTC and MSFT cannot be attributed simply to pooling, to movement prediction alone, or to tie filtering alone. At

the same time, the study does not fully identify the structural cause of why these names are materially easier than AAPL, AMZN, and GOOG under the tested configurations.

Plausible explanations include differences in tick discreteness, spread regime, queue structure, volatility, event intensity, or separability of book states. The current sample is not broad enough to test these hypotheses systematically across names and dates. As a result, the cross-sectional pattern is an important empirical finding, but its structural explanation remains only partially resolved.

Model scope

A final limitation is the deliberately restrained model scope. The project uses logistic regression and HistGradientBoostingClassifier as compact linear and nonlinear benchmarks. This is appropriate for the purpose of the study, especially given the limited data coverage and the low signal-to-noise nature of short-horizon order book prediction. It also means that the report does not attempt to establish an upper bound on predictive performance across richer model families.

This limitation is closely connected to the data constraint. With broader historical coverage, it would be more meaningful to evaluate richer tabular models, probability calibration procedures, sequence models, or more systematic hyperparameter search. Under the present sample coverage, however, expanding model complexity would add limited inferential value and would make the conclusions harder to interpret. The project is therefore informative about a disciplined tabular pipeline under explicit constraints, but not about the full frontier of short-horizon LOB prediction methods.

Scope of conclusions

Taken together, these limitations define the proper reading of the report. Many of them are deliberate scope boundaries rather than shortcomings of implementation. The project supports a careful study of short-horizon order book prediction within a controlled sample and a well-specified engineering framework. Its conclusions are meaningful within that setting, but they are not intended to extend beyond it without additional data, broader empirical coverage, and further investigation.

XI. Extensions and Future Work

Broader data coverage

The most direct extension of MicroAlpha Engine is broader data coverage. The current study is built on five tickers from a single trading day, which is sufficient for the present analysis but limits the scope of inference. Extending the dataset across more dates, more names, and a wider range of market conditions would strengthen the empirical side of the project substantially. In particular, broader coverage would make it possible to distinguish more clearly between sample-specific effects and more persistent microstructure regularities.

Cross-sectional heterogeneity

A second extension concerns the cross-sectional heterogeneity identified in the main results. One of the central findings of the project is that predictability differs sharply across names, with INTC and MSFT materially easier than AAPL, AMZN, and GOOG under several task formulations. The project narrows this phenomenon but does not fully explain it. A natural next step would therefore be a more systematic cross-sectional study linking predictive performance to observable microstructure characteristics such as spread regime, tick discreteness, queue structure, volatility, and event intensity. This would turn an empirical observation from the current report into a more explicit research question in its own right.

Task and horizon analysis

A third extension is broader task and horizon analysis. The report shows that horizon choice and tie treatment materially affect the interpretation of the problem. A more extensive study could examine how these effects evolve across a wider range of event horizons and how the relative difficulty of movement, conditional direction, and unconditional direction changes across names and market regimes. This would provide a more complete view of how target construction interacts with short-horizon order book dynamics.

Modeling extensions

A fourth extension concerns the modeling layer. The present project deliberately restricts model scope to logistic regression and HistGradientBoostingClassifier in order to keep the study controlled and interpretable. With broader data coverage, this could be extended to a wider family of tabular methods, probability calibration procedures, and thresholding or decision rules. The appropriate next step would not be model expansion for its own sake, but rather incremental extensions that preserve the interpretability and discipline of the existing pipeline while testing whether the main empirical conclusions remain stable under richer specifications.

Future research path

Taken together, these extensions suggest a clear path forward. The immediate next steps are broader empirical coverage, more systematic cross-sectional analysis, and further examination of task formulation. Longer-term extensions would build on the same engineering framework while moving toward a wider set of empirical microstructure questions.

XII. Conclusion

MicroAlpha Engine was developed as a focused study of short-horizon limit order book prediction within a modular and reproducible research framework. Using LOBSTER sample data for five NASDAQ names, it examined whether near-term midprice direction could be inferred from contemporaneous order book state and recent event flow. The resulting system combines C++ feature computation, Python orchestration, explicit task design, time-respecting train-test splitting, pooled multi-ticker evaluation, and structured artifact generation within a single pipeline.

Empirically, the main pooled reference specification produced a clear predictive signal, with ROC AUC of 0.679 for logistic regression and 0.686 for HistGradientBoostingClassifier. These aggregate results, however, masked substantial cross-sectional heterogeneity. AAPL, AMZN, and GOOG remained close to weak predictive regimes, while INTC and MSFT were materially stronger under the same pooled-trained setup. Additional diagnostic runs showed that this heterogeneity is closely linked to task formulation: removing zero forward returns turns the main directional setup into prediction conditional on movement, and this affects some names much more than others. Pooling helps, but not uniformly, and the strongest cross-sectional effects cannot be reduced to pooling alone, movement prediction alone, or tie filtering alone.

The model-interpretation results were economically coherent. Across coefficients and permutation importance, both models relied primarily on a stable core of best-level queue imbalance, microprice deviation, and short-horizon accumulated OFI, while the nonlinear model drew additional value from depth imbalance, volatility, and related state variables. This concentration of importance supports the view that the models are capturing recognizable local book dynamics rather than arbitrary feature interactions.

The engineering contribution is equally central to the project. The full production feature set is implemented in C++, exposed through pybind11, controlled through typed configuration, and integrated into a Python pipeline with tests, Docker support, and per-run artifacts. The result is not only an empirical analysis, but also a compact research-engineering system designed to make the workflow explicit, inspectable, and reproducible.

The scope of the report remains intentionally bounded. The data is limited, the model set is deliberately restrained, and the study stops at predictive modeling rather than extending into execution. Within those boundaries, MicroAlpha Engine provides a structured account of short-horizon order book prediction under explicit design constraints. Its main contribution is a disciplined combination of microstructure analysis, implementation quality, and empirical interpretation that can serve as a basis for broader future work.

XIII. Appendix

A. Reference Configuration

This appendix summarizes the main configuration used for the preserved reference run.

Component	Reference setting
Tickers	AAPL, AMZN, GOOG, INTC, MSFT
Date	2012-06-21
Depth	10 levels
Price scale	10,000
Task	direction
Label mode	binary_drop_ties
Horizon	H = 500 events
Split	70/30 chronological split per ticker
Models	Logistic Regression, HistGradientBoostingClassifier
HistGBDT parameters	learning_rate=0.05, max_iter=200, max_leaf_nodes=31, min_samples_leaf=50
Random seed	42

B. Feature Definitions

This appendix provides a compact reference for the fourteen production features used in the project.

Feature	Family	Description
`ofi_best`	Best-level order flow	Signed best-level order flow imbalance from price and queue updates
`ofi_best_norm`	Best-level order flow	Best-level OFI normalized by contemporaneous best bid and ask queue size
`queue_imbalance_best`	Liquidity imbalance	Relative displayed size at the best bid versus best ask
`depth_imbalance_3`	Depth imbalance	Bid-ask depth imbalance across the top 3 levels
`depth_imbalance_5`	Depth imbalance	Bid-ask depth imbalance across the top 5 levels
`depth_imbalance_10`	Depth imbalance	Bid-ask depth imbalance across the top 10 levels
`spread`	Price state	Best ask minus best bid
`microprice_deviation`	Price pressure	Microprice minus simple midprice
`ofi_roll_sum_50`	Temporal OFI	Rolling sum of best-level OFI over 50 events
`ofi_best_norm_roll_sum_10`	Temporal normalized OFI	Rolling sum of normalized OFI over 10 events
`ofi_best_norm_roll_sum_50`	Temporal normalized OFI	Rolling sum of normalized OFI over 50 events
`ofi_best_norm_roll_sum_100`	Temporal normalized OFI	Rolling sum of normalized OFI over 100 events
`midprice_vol_50`	Local volatility	Rolling standard deviation of event-to-event midprice changes over 50 events
`event_intensity_1s`	Event activity	Number of events observed in the trailing one-second window

C. Artifact Structure

The repository includes one preserved reference run as a public example artifact. Each completed experiment is written to a timestamped run directory containing configuration files, metrics, diagnostics, figures, fitted models, and logs. The purpose of this structure is to make each run inspectable after execution without requiring the experiment to be rerun.

A typical run directory contains the following structure:

Location	Artifact	Purpose
root	`config.json`	Serialized experiment configuration
root	`metrics.json`	Aggregate model metrics for the run
root	`split_summary.json`	Pooled train-test split sizes and class balances
root	`ticker_summaries.json`	Per-ticker event counts, label summaries, tie rates, class balances, and split summaries
root	`pooled_ticker_metrics.json`	Per-ticker model performance under pooled training
root	`ticker_feature_diagnostics.json`	Per-ticker feature summary statistics
root	`logistic_coefficients.json`	Standardized logistic-regression coefficients
root	`logistic_permutation_importance.json`	Logistic-regression permutation importance
root	`hist_gbdt_permutation_importance.json`	HistGradientBoosting permutation importance
tables/	`logistic_permutation_importance.csv`	Tabular logistic-regression permutation importance
tables/	`hist_gbdt_permutation_importance.csv`	Tabular HistGradientBoosting permutation importance
tables/	`pooled_ticker_metrics.csv`	Tabular per-ticker pooled evaluation
figures/	`confusion_matrix_comparison.png`	Confusion matrices for the reference models
figures/	`feature_importance_logistic.png`	Logistic-regression permutation-importance plot
figures/	`feature_importance_hist_gbdt.png`	HistGradientBoosting permutation-importance plot
figures/	`model_comparison_roc_auc.png`	Aggregate ROC AUC model comparison
figures/	`per_ticker_auc_comparison.png`	Per-ticker ROC AUC comparison under pooled training
figures/	`roc_comparison.png`	ROC curves for the reference models
figures/	`score_distribution_comparison.png`	Predicted-score distributions by class
models/	`logistic.joblib`	Serialized fitted logistic-regression model
models/	`hist_gbdt.joblib`	Serialized fitted HistGradientBoosting model
logs/	`run.log`	Execution log for the run

This structure is consistent with the preserved reference configuration, which stores the five LOBSTER sample tickers, depth, price scale, task, label mode, feature windows, split configuration, and model parameters in `config.json`. The run also stores aggregate metrics, pooled split summaries, per-ticker metrics, and ticker-level summaries as separate JSON outputs.

D. Additional Diagnostics

The following figures provide secondary diagnostics for the preserved reference run. They are included to document model behavior beyond the main reported metrics, but they are not used as primary evidence in the report.

This figure compares the ROC curves of logistic regression and HistGradientBoostingClassifier in the pooled reference run. Both models remain above the diagonal baseline, with HistGradientBoosting slightly ahead in aggregate ROC AUC. The corresponding aggregate metrics are stored in `metrics.json`.

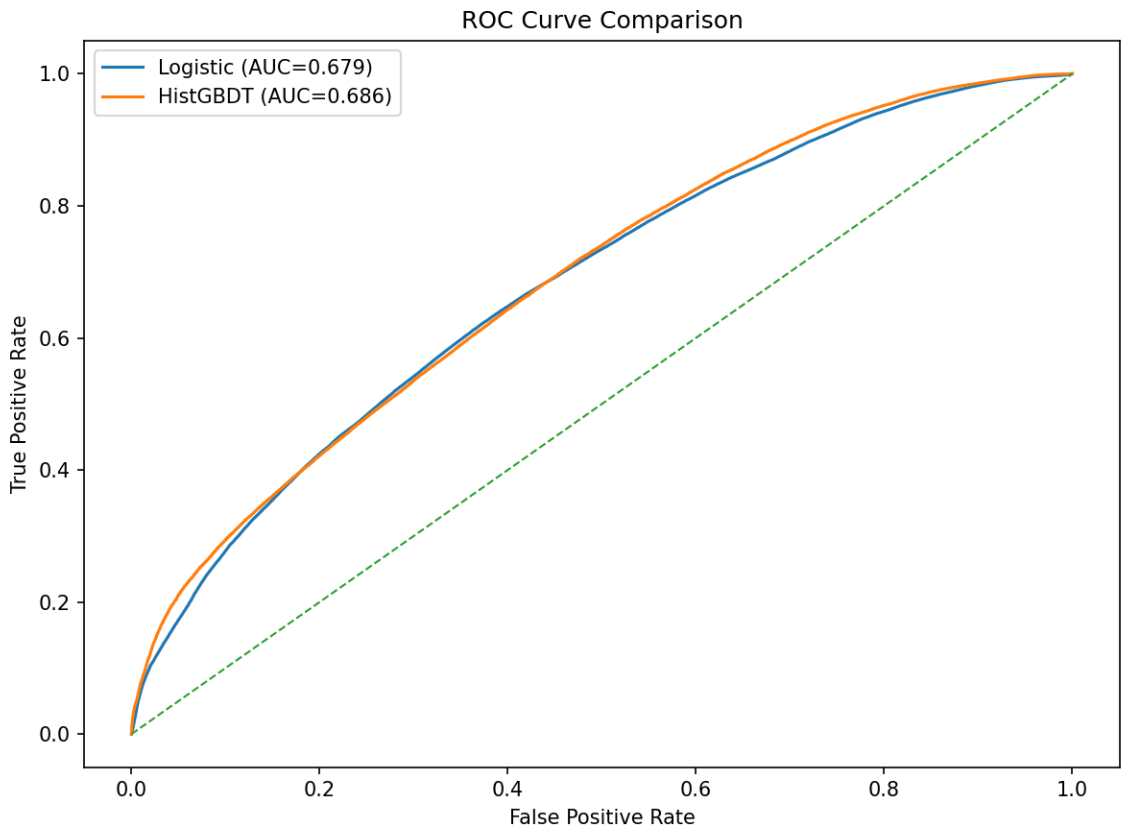

Figure D.1. ROC curve comparison

This figure shows the predicted-score distributions by true class for both models. It provides additional context on how much class separation the models achieve beyond the scalar ROC AUC values. The distributions are especially useful for seeing that the problem remains noisy despite aggregate discrimination above random.

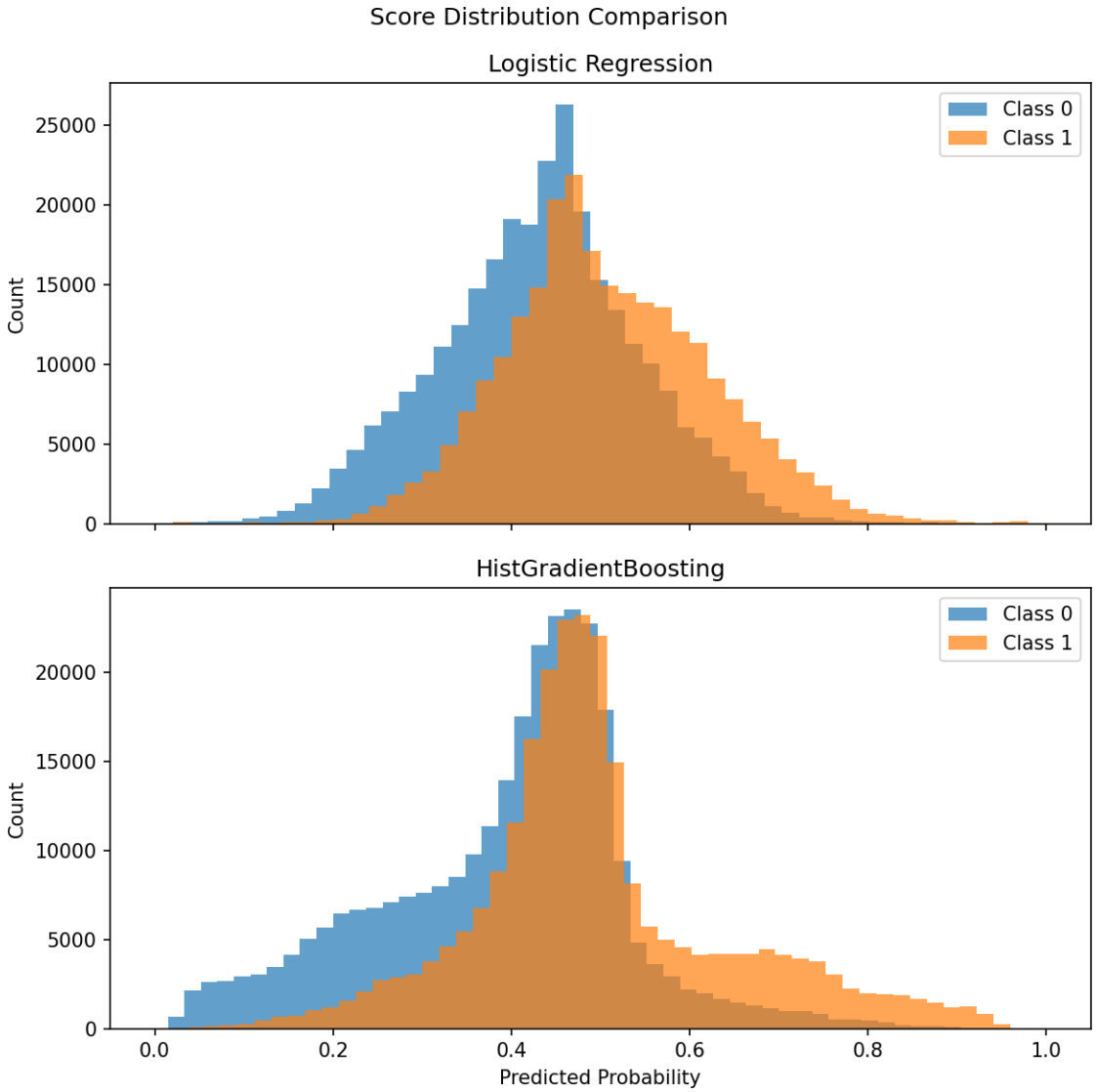

Figure D.2. Score distribution comparison

The figure reports threshold-based classification counts for logistic regression and HistGradientBoostingClassifier on the pooled test set. It is included as a secondary diagnostic, as the main evaluation in the report is based primarily on ROC AUC rather than a fixed classification threshold.

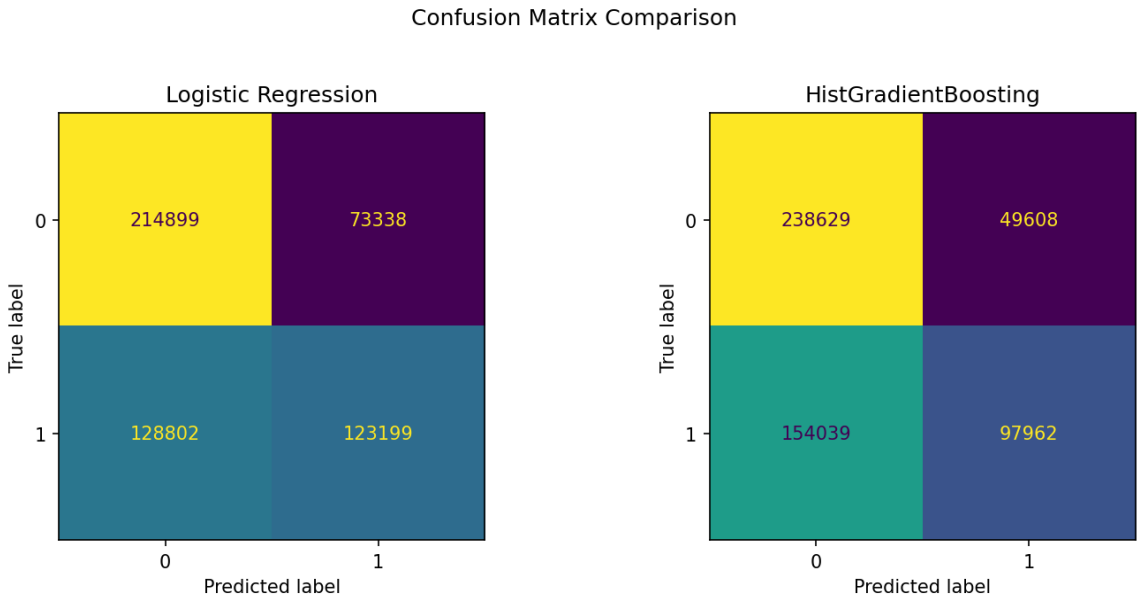

Figure D.3. Confusion matrix comparison